

Product Requirements Document

Role & Objective:

You are a **Senior Frontend Architect and UI/UX Designer**. Your primary objective is to independently **design, scaffold, and implement a complete, production-ready frontend web application**. This application will interact exclusively with a public mock API. The output should be well-structured and include **full source code suitable for hosting on GitHub**. This project is intended as an educational showcase for the capabilities of agentic AI.

1. Non-Negotiable Constraints (Guardrails):

- **Sole Data Source:** The **JSONPlaceholder API** (<https://jsonplaceholder.typicode.com/>) is the **only permitted source for all data**.
 - **No Backend:** Do not create or rely on any backend services, custom API routes, local databases, or server-side persistence layers.
 - **Direct API Interaction:** All data operations must interact **directly** with JSONPlaceholder:
 - GET requests for data retrieval.
 - POST, PUT, DELETE requests for simulated state mutations.
 - **Mutation Handling:** As JSONPlaceholder does not persist mutations:
 - Implement **optimistic UI updates** to reflect changes immediately on the frontend.
 - Ensure clear user feedback (e.g., visual indication, error messages) for failed mutation requests.
-

2. Product & UX Strategy (Autonomous Decisions Required):

- **Application Concept:** Analyze the available JSONPlaceholder resources (Users, Posts, Comments, Todos, Albums, Photos) and autonomously determine the **most logical and user-friendly application concept**. Examples: a unified dashboard, a content management interface, a social-style feed, or an admin console.
 - **User Experience Definition:** Define a primary user persona, core user flows, and the application's information architecture based on your chosen concept.
 - **Technology Stack Selection:** Choose an appropriate:
 - **Frontend Framework:** e.g., React, Next.js, Vue.
 - **Styling Solution:** e.g., CSS Modules, Tailwind CSS, Styled Components.
 - **State Management Approach:** e.g., React Context, Redux, Zustand, Vuex, Pinia, or a server-state library like TanStack Query.
 - **Decision Rationale:** For all technology choices and architectural decisions, provide **brief, high-level justifications (1–3 sentences each)**. Do not include extended theoretical explanations unless explicitly requested.
-

3. Technical Execution Requirements:

- **Architecture & Components:** Analyze the API endpoints and define a clear component and state architecture.
 - **Project Scaffolding:** Set up the project using a modern build tool (e.g., Vite, Create React App, Next.js CLI) and install necessary dependencies.
 - **UI Implementation:**
 - Build a **clean, modern, and production-ready UI** that prioritizes usability and clarity. Enterprise-level visual polish or custom design systems are **not required**.
 - Ensure the UI is **responsive** (adapts to various screen sizes).
 - Implement robust **loading states, error handling, and empty states** for all API interactions.
 - Integrate the optimistic UI updates for mutations.
 - **Basic Quality:** Apply basic accessibility practices (e.g., semantic HTML, appropriate ARIA attributes where necessary) and ensure a consistent visual design.
 - **Validation:** The application's correctness will be validated through **manual testing** of key user flows by the agent.
-

4. Deployment Readiness (Google Cloud Run):

- **Containerization:** The final application must be containerized and ready for deployment to Google Cloud Run.
 - **Optimized Dockerfile:** Generate a **production-optimized Dockerfile** that:
 - Exposes the **correct port (8080)** as required by Cloud Run.
 - Aims for efficient image size.
 - Uses a lightweight web server (like Nginx) for serving static assets if a static site generator is chosen.
 - Utilizes an optimized Node.js production setup if SSR/hybrid rendering is chosen.
 - **Deployment Clarity:** Include brief instructions or explanations on how the container is expected to run on Google Cloud Run.
-

5. Final Deliverables:

Provide the following:

1. **High-level architectural overview** of the chosen application concept.
2. **Key technology stack choices** with their brief justifications.
3. An outline of the **project structure**.
4. **Full source code** for the frontend application, organized logically for GitHub hosting (e.g., in appropriate folders like src/components, src/pages, etc.).
5. The **Dockerfile**.
6. Any necessary **build and run instructions** for local development and container deployment.

7. **References to external resources** that explain major architectural patterns employed (e.g., links to articles, documentation on patterns like optimistic UI, component-based architecture, state management patterns).
-